

SwapBuds Automation & DevOps Setup Guide

Complete Pre-Launch Automation Configuration

EXECUTIVE SUMMARY

As a solo developer, automation is your force multiplier. This guide outlines the exact tools, processes, and configurations you need to:

1. **Prevent bugs** from reaching production
2. **Catch errors** before users do
3. **Monitor health** 24/7 without manual checks
4. **Scale reliably** without heroic efforts
5. **Sleep soundly** knowing systems are protected

Total setup time: 12-15 hours

Time saved per month: 20+ hours

Reliability improvement: 10x

SECTION 1: CI/CD PIPELINE (GitHub Actions)

Why GitHub Actions?

- ✓ Free for public repos, generous free tier for private
- ✓ Native to GitHub, no third-party integration needed
- ✓ Triggers on PR, push, schedule, manual
- ✓ Can deploy directly from GitHub
- ✓ Already mentioned in your [DEPLOYMENT.md](#)

Backend CI/CD Workflow

File location: `.github/workflows/backend-ci.yml`

```
name: Backend CI/CD
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: swapbuds_test
        options: &gt;-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432
      redis:
        image: redis:7
        options: &gt;-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 6379:6379
```

```

steps:
  - uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: '18'
      cache: yarn

  - name: Install dependencies
    run: yarn install --frozen-lockfile

  - name: Generate Prisma Client
    run: yarn prisma generate

  - name: Run database migrations
    run: yarn prisma migrate deploy
    env:
      DATABASE_URL: postgresql://postgres:postgres@localhost:5432/swapbuds_test

  - name: Run linter
    run: yarn lint

  - name: Type check
    run: yarn tsc --noEmit

  - name: Run tests
    run: yarn test:cov
    env:
      DATABASE_URL: postgresql://postgres:postgres@localhost:5432/swapbuds_test
      REDIS_HOST: localhost
      REDIS_PORT: 6379
      JWT_SECRET: test-secret
      JWT_REFRESH_SECRET: test-refresh-secret

  - name: Upload coverage to Codecov
    uses: codecov/codecov-action@v3
    with:
      files: ./coverage/lcov.info
      fail_ci_if_error: true

deploy:
  needs: test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4

    - name: Deploy to Vercel
      uses: amondnet/vercel-action@v25
      with:
        vercel-token: ${ secrets.VERCEL_TOKEN }
        vercel-org-id: ${ secrets.VERCEL_ORG_ID }
        vercel-project-id: ${ secrets.VERCEL_BACKEND_PROJECT_ID }
        vercel-args: --prod

```

What this does:

- Runs on every push to main/develop
- Runs on every PR to main/develop
- Spins up PostgreSQL and Redis services
- Installs dependencies
- Generates Prisma client
- Runs database migrations
- Lints code
- Type-checks TypeScript
- Runs 484 tests with coverage
- Uploads coverage to Codecov
- Deploys to production if main branch and tests pass

Setup time: 1 hour

Frontend CI/CD Workflow

File location: `.github/workflows/frontend-ci.yml`

```
name: Frontend CI/CD
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'
          cache: yarn

      - name: Install dependencies
        run: yarn install --frozen-lockfile

      - name: Run linter
        run: yarn lint

      - name: Type check
        run: yarn tsc --noEmit

      - name: Build
        run: yarn build
        env:
          NEXT_PUBLIC_API_URL: ${ secrets.NEXT_PUBLIC_API_URL }

      - name: Run E2E tests
        run: yarn test:e2e

      - name: Run Lighthouse CI
        uses: treosh/lighthouse-ci-action@v10
        with:
          urls: |
            http://localhost:3000
            http://localhost:3000/login
            http://localhost:3000/items
          uploadArtifacts: true

  deploy:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.VERCEL_ORG_ID }
          vercel-project-id: ${ secrets.VERCEL_FRONTEND_PROJECT_ID }
          vercel-args: --prod
```

What this does:

- Runs tests on every PR and push
- Lints code
- Type-checks TypeScript
- Builds Next.js production bundle
- Runs E2E tests
- Runs Lighthouse CI (performance, accessibility, SEO)
- Deploys to production if main branch and all tests pass

Setup time: 1 hour

Branch Protection Rules

In GitHub Settings → Branches → Main:

- [x] Require a pull request before merging
- [x] Require approvals: 1 (you review your own PRs from develop)
- [x] Require status checks to pass before merging:
 - [x] backend-ci / test
 - [x] backend-ci / lint
 - [x] frontend-ci / test
 - [x] frontend-ci / lint
- [x] Require branches to be up to date before merging
- [x] Require conversation resolution before merging
- [x] Restrict who can push to matching branches (only you)
- [x] Require signed commits (recommended but optional)

Purpose: Ensures no broken code reaches production. Forces you to go through develop branch first.

Workflow for pushing code:

1. Create feature branch from develop: `git checkout -b feature/item-management`
2. Push to GitHub: `git push origin feature/item-management`
3. Create PR on develop (not main)
4. GitHub Actions runs tests automatically
5. If tests pass, merge to develop
6. When ready for production, create PR: develop → main
7. GitHub Actions runs again
8. If tests pass, merge to main
9. Automatic deploy to production happens
10. Monitoring alerts activate

Setup time: 30 minutes

SECTION 2: DEPENDENCY MANAGEMENT

Dependabot Automation

File location: `.github/dependabot.yml`

```
version: 2
updates:
  - package-ecosystem: npm
    directory: "/backend"
    schedule:
      interval: weekly
      day: monday
      time: "03:00"
    reviewers:
      - "yourusername"
    assignees:
      - "yourusername"
    pull-request-branch-name:
      separator: "/"
    allow:
      - dependency-type: direct
      - dependency-type: indirect
    labels:
      - dependencies
    rebase-strategy: auto

  - package-ecosystem: npm
    directory: "/frontend"
```

```
schedule:
  interval: weekly
  day: monday
  time: "04:00"
reviewers:
  - "yourusername"
assignees:
  - "yourusername"
pull-request-branch-name:
  separator: "/"
allow:
  - dependency-type: direct
  - dependency-type: indirect
labels:
  - dependencies
rebase-strategy: auto
```

What this does:

- Every Monday, Dependabot checks for new dependency versions
- Creates automatic PRs for updates
- Can be configured to auto-merge safe updates (patch versions)

Setup time: 30 minutes

Security Scanning

In GitHub Settings → Security → Secret scanning:

- [x] Enable secret scanning
- [x] Enable push protection (prevents commits with secrets)

In GitHub Settings → Code security → Dependabot:

- [x] Enable Dependabot alerts
- [x] Enable Dependabot security updates
- [x] Enable Dependabot version updates

What this does:

- GitHub automatically detects exposed secrets
- Prevents committing secrets to repo
- Alerts you to vulnerable dependencies
- Auto-creates PRs to fix vulnerabilities

Setup time: 15 minutes

SECTION 3: CODE COVERAGE TRACKING

Codecov Integration

Codecov automatically tracks code coverage over time, alerting if coverage drops.

Setup:

1. Sign up at codecov.io
2. Connect GitHub account
3. Codecov automatically picks up coverage from GitHub Actions
4. In repo settings, enable coverage requirements:
 - Minimum coverage: 80%
 - Fail PR if coverage drops below threshold

What this does:

- Tracks test coverage trends over time
- Prevents PRs that reduce coverage
- Alerts if a PR decreases coverage

- Helps maintain code quality

Cost: Free for public repos

Setup time: 30 minutes

SECTION 4: ERROR MONITORING (Sentry)

Sentry catches errors before users report them. Critical for solo developer.

Backend Sentry Setup

Installation:

```
# Backend
yarn add @sentry/node @sentry/tracing

# Frontend
yarn add @sentry/react @sentry/tracing
```

Backend initialization (main.ts):

```
import * as Sentry from "@sentry/node";
import { nodeProfilingIntegration } from "@sentry/profiling-node";

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  integrations: [
    nodeProfilingIntegration(),
    new Sentry.Integrations.OnUncaughtException(),
    new Sentry.Integrations.OnUnhandledRejection(),
  ],
  environment: process.env.NODE_ENV,
  tracesSampleRate: 1.0,
  enabled: process.env.NODE_ENV === 'production',
});

// Attach Sentry to Express
app.use(Sentry.Handlers.requestHandler());
app.use(Sentry.Handlers.errorHandler());
```

Frontend initialization (pages/_app.tsx):

```
import * as Sentry from "@sentry/react";

Sentry.init({
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
  environment: process.env.NODE_ENV,
  tracesSampleRate: 1.0,
  enabled: process.env.NODE_ENV === 'production',
});

export default Sentry.withProfiler(MyApp);
```

Environment variables needed:

```
# Backend
SENTRY_DSN=https://...@sentry.io/...
SENTRY_AUTH_TOKEN=sentrys...

# Frontend
NEXT_PUBLIC_SENTRY_DSN=https://...@sentry.io/...
```

Sentry Alerts Configuration

In Sentry dashboard:

1. **New Issue Alert** → Notify for every new error
2. **Regression Alert** → Notify if error happens again after being fixed
3. **Frequency Alert** → Notify if error rate spikes
4. **Environment Alert** → Different alerts for dev/staging/prod

Alert channels:

- Email (primary)
- Slack (if you have Slack workspace)
- PagerDuty (if on paid plan)

What to monitor:

- Error rate > 1% of requests
- Specific endpoints with errors
- Database connection errors
- Authentication errors
- File upload errors

Setup time: 2 hours

Cost: \$26/month Team plan (includes session replay)

SECTION 5: UPTIME MONITORING (BetterStack)

BetterStack monitors your API 24/7 from multiple locations and alerts you if down.

Setup:

1. Sign up at betterstack.com
2. Create HTTP monitor for main API endpoint
3. Create HTTP monitor for frontend
4. Set check frequency: 1 minute
5. Configure alert channels: Email + SMS if possible

Example monitor:

- **URL:** <https://api.swapbuds.ro/health>
- **Method:** GET
- **Expected status:** 200
- **Check interval:** 1 minute
- **Timeout:** 10 seconds
- **Locations:** Europe 1, Europe 2, US East, US West

Backend health endpoint:

```
// In NestJS controller
@Get('/health')
@HttpCode(HttpStatus.OK)
async health() {
  // Quick checks to verify system is healthy
  const dbOk = await this.db.isConnected();
  const redisOk = await this.redis.ping();

  if (!dbOk || !redisOk) {
    throw new ServiceUnavailableException('Health check failed');
  }

  return {
    status: 'ok',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
  };
}
```

```
};  
}
```

What this does:

- Pings your API every minute from multiple locations
- If API doesn't respond with 200, alerts you immediately
- Provides uptime percentage (target 99.9%)
- Status page for users to check system status

Cost: \$10/month for uptime monitoring

Setup time: 1 hour

SECTION 6: METRICS & LOGS (Grafana Cloud)

Grafana Cloud provides free metrics, logs, and traces storage.

Setup:

1. Sign up at grafana.com (free tier)
2. Create data source connections:
 - **Prometheus** - for metrics
 - **Loki** - for logs
 - **Tempo** - for traces (optional)

Application metrics to track:

Backend metrics (using prom-client):

```
import promClient from 'prom-client';  
  
// Create metrics  
const httpRequestDuration = new promClient.Histogram({  
  name: 'http_request_duration_seconds',  
  help: 'Duration of HTTP requests in seconds',  
  labelNames: ['method', 'route', 'status_code'],  
  buckets: [0.1, 0.5, 1, 2, 5],  
});  
  
const dbQueryDuration = new promClient.Histogram({  
  name: 'db_query_duration_seconds',  
  help: 'Duration of database queries',  
  labelNames: ['query_type'],  
});  
  
const activeConnections = new promClient.Gauge({  
  name: 'active_connections',  
  help: 'Number of active database connections',  
});  
  
// Track in middleware  
app.use((req, res, next) => {  
  const start = Date.now();  
  res.on('finish', () => {  
    const duration = (Date.now() - start) / 1000;  
    httpRequestDuration  
      .labels(req.method, req.route?.path || req.path, res.statusCode)  
      .observe(duration);  
  });  
  next();  
});  
  
// Expose metrics endpoint  
app.get('/metrics', (req, res) => {  
  res.set('Content-Type', promClient.register.contentType);  
  res.end(promClient.register.metrics());  
});
```

Logs (structured logging):

```
// Use Winston or Pino for structured logging
import winston from 'winston';

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  defaultMeta: { service: 'swapbuds-api' },
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' }),
  ],
});

// Log all requests
app.use((req, res, next) => {
  logger.info('Incoming request', {
    method: req.method,
    url: req.url,
    ip: req.ip,
  });
  next();
});
```

Key dashboards to create:

1. Request rate and latency
2. Error rate by endpoint
3. Database connection pool usage
4. Cache hit rate
5. Active user sessions
6. API uptime percentage

Setup time: 2 hours

Cost: Free tier with limitations, sufficient for MVP

SECTION 7: PERFORMANCE MONITORING (Lighthouse CI)

Already integrated in frontend CI/CD workflow. Automatically runs on every PR.

Configuration:

File: `.github/workflows/frontend-ci.yml` (already included above)

Lighthouse checks:

- Performance: Measure page load time, Core Web Vitals
- Accessibility: WCAG compliance
- Best Practices: Security, browser compatibility
- SEO: Meta tags, mobile-friendly

Setup: Already included in frontend CI workflow (0 additional time)

Benefit: Prevents performance regressions, ensures accessibility standards

SECTION 8: BACKUP AUTOMATION

Neon Database Backups

Already built-in! Neon automatically backs up daily with 30-day retention.

What to do:

- Verify backups are enabled in Neon dashboard
- Test restoration monthly (create test environment from backup)
- Document restoration procedure
- Set reminder to test restoration every 30 days

Cloudinary Backups

Already built-in! Cloudinary has automatic regional backups.

What to do:

- Enable versioning in Cloudinary (allows rollback)
- Set auto-delete old versions after 30 days
- Periodically export media library (manual export in dashboard)

Application Backups

Code: Backed up automatically by GitHub (infinite history)

Environment: Document in encrypted file or password manager

User data: Automatically backed up with database

Backup retention policy:

- Daily backups: Retain 30 days (automatic)
- Weekly backups: Retain 90 days (on demand)
- Monthly backups: Retain 1 year (on demand)

Setup time: 1 hour (mostly documentation)

SECTION 9: DISASTER RECOVERY PLAN

As solo developer, you need quick recovery procedures.

Recovery Time Objectives (RTO)

- **Database down:** 4 hours maximum
- **API down:** 1 hour maximum
- **Frontend down:** 30 minutes maximum
- **Complete outage:** 8 hours maximum

Recovery Point Objectives (RPO)

- **Database:** 24 hours (daily backups)
- **User data:** 24 hours (database backups)
- **Files:** 7 days (Cloudinary versioning)

Procedures to document:

1. Database Recovery

```
Step 1: Identify issue (Sentry/BetterStack alert)
Step 2: Check Neon dashboard - backups available?
Step 3: Create new database instance from backup
Step 4: Point backend to new database
Step 5: Verify data integrity
Step 6: Re-enable services
```

2. API Failure Recovery

```
Step 1: Check GitHub Actions - deployment failed?
Step 2: Check Vercel/Railway - service down?
Step 3: Automatic rollback (if deployed recently)
Step 4: OR manual rollback: revert last commit, redeploy
Step 5: Verify API responding
Step 6: Check Sentry for errors
```

3. Frontend Failure Recovery

```
Step 1: Check Vercel build logs
Step 2: Check if build failed or deployment failed
Step 3: Fix issue or rollback
```

Step 4: Redeploy
Step 5: Clear browser cache, test

Setup time: 2 hours (mostly documentation and testing)

SECTION 10: RUNBOOK (Common Issues)

Create a runbook for quick troubleshooting.

Issue: High Error Rate (>5%)

1. Check Sentry dashboard - what errors?
2. Check recent deployments - did we just deploy?
3. If recent deploy, check git logs: `git log -1 --oneline`
4. Compare with previous version
5. Fix issue and redeploy OR rollback if unsure
6. Monitor error rate for 30 minutes

Issue: Slow API Response (>500ms)

1. Check slow query log: Sentry or Grafana
2. Identify slow query
3. Add database index if missing
4. Or optimize query (add caching, select fewer fields)
5. Deploy fix
6. Monitor response time for 1 hour

Issue: Database Connection Pool Exhausted

1. Check active connections in Grafana
2. Find long-running queries in logs
3. Increase pool size or optimize queries
4. Redeploy
5. Monitor connections for 1 hour

Issue: Authentication Failures Spiking

1. Check Sentry for error details
2. Verify JWT secrets are correct
3. Check if token expiration is too short
4. If user impact, send status update
5. Fix issue and redeploy
6. Monitor auth failures

Issue: File Upload Failures

1. Check Cloudinary status page
2. Verify API key is correct
3. Check file size limits
4. Check file format restrictions
5. Verify storage quota not exceeded
6. If Cloudinary down, inform users

Setup time: 1 hour (mostly documentation)

SECTION 11: ALERTING CONFIGURATION

Email Alerts (Primary - Always use)

Critical Alerts (immediate action needed):

- API down (uptime check fails)
- Database down (Sentry detects connection errors)
- Error rate > 5%
- Deployment failed

Important Alerts (within 1 hour):

- Error rate > 2%
- Response time > 500ms
- Database CPU > 80%
- Cache miss rate > 30%

Info Alerts (daily digest):

- New feature releases (Dependabot PRs)
- Weekly coverage report
- Weekly uptime summary
- Monthly cost summary

Slack Alerts (Optional - if you use Slack)

Setup:

1. Create #swapbuds-alerts channel in Slack
2. Connect Sentry to Slack
3. Connect BetterStack to Slack
4. Connect GitHub Actions to Slack

Alert rules:

- Post critical errors to #swapbuds-alerts
- Post deployment status to #swapbuds-alerts
- Post uptime alerts to #swapbuds-alerts

Cost: Slack requires paid plan for advanced integrations

SECTION 12: IMPLEMENTATION TIMELINE

Week 1-2: Essential Setup

- [] GitHub Actions (backend + frontend CI/CD) - 2 hours
- [] Branch protection rules - 0.5 hours
- [] Dependabot configuration - 0.5 hours
- [] GitHub secret scanning - 0.25 hours
- **Total: 3.25 hours**

Week 3-4: Monitoring & Alerting

- [] Sentry setup (backend + frontend) - 2 hours
- [] BetterStack uptime monitoring - 1 hour
- [] Email alert configuration - 0.5 hours
- **Total: 3.5 hours**

Week 5-6: Metrics & Visibility

- ☐ Grafana Cloud setup - 2 hours
- ☐ Application metrics instrumentation - 2 hours
- ☐ Lighthouse CI integration - 0.5 hours (already done)
- **Total: 4.5 hours**

Week 7-8: Disaster Recovery

- ☐ Backup strategy documentation - 1 hour
- ☐ Recovery procedures documentation - 1 hour
- ☐ Disaster recovery testing - 2 hours
- ☐ Runbook creation - 1 hour
- **Total: 5 hours**

Week 9: Final Verification

- ☐ Test all alert channels - 1 hour
- ☐ Perform disaster recovery drill - 2 hours
- ☐ Document lessons learned - 1 hour
- **Total: 4 hours**

Grand Total: ~20 hours (spread across 9 weeks)

SECTION 13: COST BREAKDOWN

Tool	Cost	Required?	Purpose
GitHub Actions	Free	✓ Yes	CI/CD automation
Codecov	Free	⚠ Optional	Coverage tracking
Sentry	\$26/month	✓ Yes	Error monitoring
BetterStack	\$10/month	✓ Yes	Uptime monitoring
Grafana Cloud	Free	⚠ Optional	Metrics & dashboards
Total	\$36/month		Core monitoring

Optional add-ons:

- Slack integrations: Requires Slack Pro (\$9/month)
- PagerDuty: \$9-99/month (overkill for solo dev)
- Datadog: \$15+/month (overkill for MVP)

SECTION 14: BEFORE LAUNCH CHECKLIST

Automation Readiness

- ☐ GitHub Actions workflows tested
- ☐ Backend CI pipeline passing
- ☐ Frontend CI pipeline passing
- ☐ Auto-deploy working (main → production)
- ☐ Branch protection rules enforced

Monitoring Readiness

- ☐ Sentry configured for backend
- ☐ Sentry configured for frontend
- ☐ Sentry dashboards accessible
- ☐ BetterStack monitors configured
- ☐ Test alerts working (received test alerts)

Alerting Readiness

- ☐ Email alerts configured
- ☐ Slack alerts configured (optional)
- ☐ Alert thresholds set appropriately
- ☐ Contact list updated
- ☐ On-call rotation if team, or just you if solo

Backup & Recovery Readiness

- ☐ Database backup tested
- ☐ Restore procedure documented
- ☐ Disaster recovery plan documented
- ☐ Runbook created and documented
- ☐ Team (or just you) trained on procedures

Performance Readiness

- ☐ Lighthouse CI running on every PR
- ☐ Performance budgets set
- ☐ Core Web Vitals tracked
- ☐ Database metrics baseline recorded
- ☐ API response time baseline recorded

SECTION 15: POST-LAUNCH MONITORING

Daily (First Week)

- ☐ Check error rate in Sentry
- ☐ Check uptime in BetterStack
- ☐ Check GitHub Actions for deployment status
- ☐ Monitor social media for reported issues
- ☐ Fix critical issues immediately

Weekly (First Month)

- ☐ Review Sentry error patterns
- ☐ Review Grafana metrics dashboards
- ☐ Check Lighthouse CI scores
- ☐ Review code coverage trends
- ☐ Plan performance optimizations if needed

Monthly (Ongoing)

- ☐ Review uptime percentage
- ☐ Analyze deployment success rate
- ☐ Test disaster recovery procedures
- ☐ Review cost trends
- ☐ Update runbook with new issues encountered
- ☐ Security audit of logs and errors

CRITICAL SUCCESS FACTORS

1. **Don't skip monitoring** - As solo developer, you're on call 24/7
2. **Automate repetitive tasks** - Manual deployment is error-prone
3. **Test recovery procedures** - Don't discover issues during incident
4. **Document everything** - Future you will forget the details
5. **Set reasonable alert thresholds** - Alert fatigue is real
6. **Review logs regularly** - Catch patterns before they become problems

NEXT STEPS

1. **This week:** Set up GitHub Actions (3 hours)
2. **Next week:** Set up Sentry and BetterStack (2.5 hours)
3. **Week 3:** Document backup/recovery procedures (2 hours)
4. **Week 4:** Test everything and create runbook (2 hours)

Total time investment: ~9.5 hours

Benefit: Sleep soundly knowing systems are monitored 24/7

You've built 484 tests and a production-ready backend. This automation setup is entirely achievable. Let's do this! 🚀

Version 1.0

Last Updated: November 29, 2025

For: Darius Bere, SwapBuds Solo Developer